Introduction to GraphQL with Node.js

Presented by:

Cristian Camilo Cortes Ortiz

Codecamp academy

Table of contents

Audience 2
Purpose 2
Introduction 2
What is GraphQL? 3
History of GraphQL 4
Type System 5
- Scalar Types 6
- Non-Null Types 6
- List Types 6
- Object Types 6
- Enumeration Types 6
- Interface Types 7
- Union Types 7
- Input Types 7

Queries 8
Mutations 9
Subscriptions 10
Variables and Aliases 12
- Variables 12
- Aliases 13

Custom Scalars 14
Dynamic Nesting 15
Building a GraphQL Server with Node.js 17
- Project Setup 17
- Schema and Resolvers 18
- Testing the Server 19

GraphQL vs REST API 20
Best Practices 21
Conclusion 22

Audience

This manual contains technical terminology and is intended for IT students, instructors, and developers who want to learn the fundamentals of GraphQL, the query language for APIs, using JavaScript in the Node.js environment. Readers are expected to be familiar with basic JavaScript, JSON, and the client-server model. Prior knowledge of REST APIs is helpful but not required.

Purpose

The purpose of this manual is to give readers a clear and practical understanding of the essential concepts of GraphQL. By the end of this document, the reader will be able to:

- Explain what GraphQL is and which problems it solves.
- Describe the GraphQL type system and define a schema.
- Write queries, mutations, and subscriptions.
- Use variables, aliases, and custom scalars.
- Build a basic GraphQL server in Node.js.
- Compare GraphQL with a traditional REST API.

Introduction

In a traditional REST architecture, each resource has its own endpoint, and the server decides what data each endpoint returns. This frequently leads to two well-known problems: over-fetching, where the client receives more data than it needs, and under-fetching, where a single screen requires several requests to different endpoints to collect all of its data.

GraphQL addresses both problems by exposing all data through a single endpoint and letting the client describe exactly what it needs. This document explores the fundamentals of the language, from its type system to queries, mutations, and subscriptions, and then applies them in a working Node.js server, providing a solid foundation for understanding the core concepts of the API query language.

What is GraphQL?

GraphQL is a query language for APIs, and a runtime for fulfilling those queries with existing data. It allows clients to request only the data they need, precisely and efficiently. Unlike traditional REST APIs, where clients usually receive predefined data in a fixed structure, GraphQL lets clients specify exactly which fields they want, reducing excess data and improving application performance.

This is achieved by defining a schema that describes the available data types and their relationships. Clients send GraphQL queries to the server, and the server executes those queries against the schema, returning only the requested data in the same shape as the query. Figure 1 illustrates this flow: the client sends a single request to one endpoint, and the server resolves each field from one or more data sources.

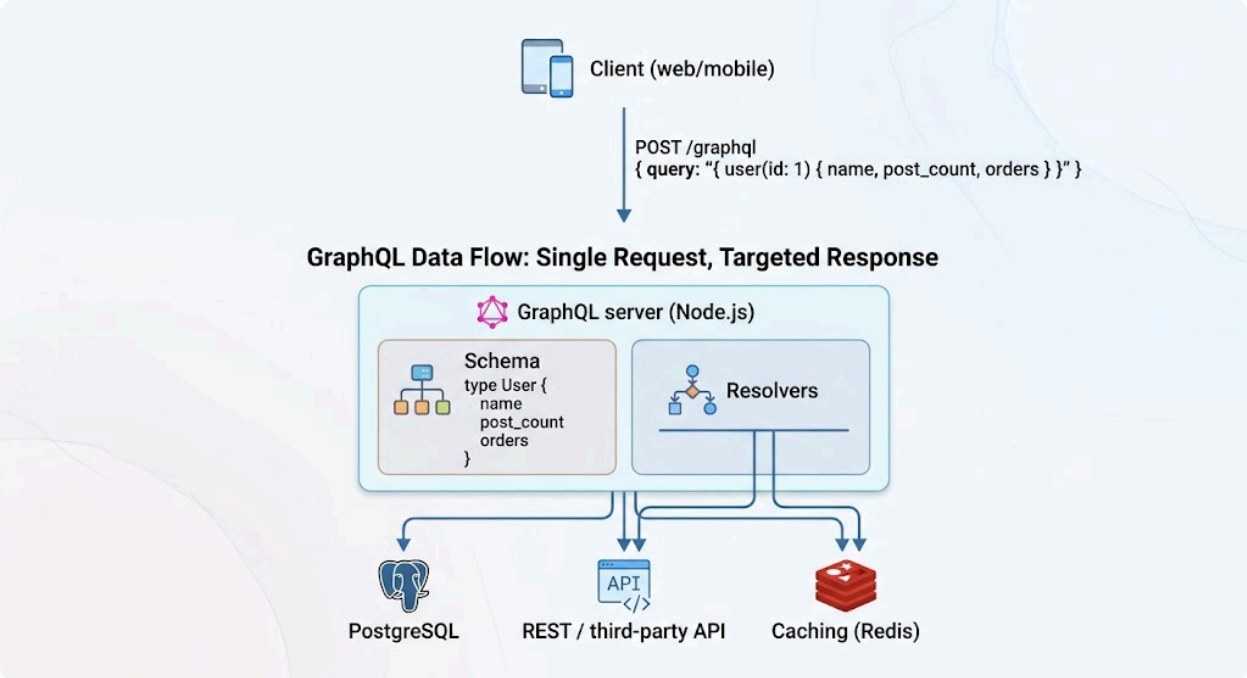


Figure 1. GraphQL request flow between the client, the server, and the data sources

This provides unprecedented flexibility in API development, enabling more efficient communication between the client and the server, and allowing front-end and back-end teams to evolve independently.

History of GraphQL

GraphQL was developed internally by Facebook's engineering team in 2012 to address the specific challenges they faced when building and maintaining complex mobile applications. Before GraphQL, Facebook used a variety of REST APIs for different parts of its application, which resulted in data overload and a fragmented development experience. With the goal of optimizing data transfer and improving development efficiency, the team created GraphQL as an innovative solution for data management.

In 2015, Facebook released GraphQL as an open-source specification, and it has since gained wide adoption in the software development community. Companies such as GitHub, Shopify, and Pinterest use it in production. Its continued evolution demonstrates its ability to address modern API development challenges and its relevance in today's technology landscape.

Type System

GraphQL has a strong type system that defines the structure and shape of data. Types represent the objects that can be queried or mutated, and they are declared in the schema using the GraphQL Schema Definition Language (SDL). Figure 2 shows a small schema that uses the main kinds of types, which are described in detail below.

```
enum Role { ADMIN STAFF GUEST }

interface Node {
  id:       ID!
}

type User implements Node {
  id:       ID!
  name:     String!
  email:    String!
  role:     Role!
  tags:     [String!]!
  bookings: [Booking!]!
}

type Booking implements Node {
  id:       ID!
  room:     String!
  nights:   Int!
}

union SearchResult = User | Booking

input CreateUserInput {
  name:     String!
  email:    String!
  role:     Role = GUEST
}
```

Figure 2. Example of a GraphQL schema written in SDL

The main types in GraphQL are:

Scalar Types

These are built-in primitive types such as String, Int, Float, Boolean, and ID (a unique identifier). They represent the leaves of a query, the fields that return a concrete value.

Non-Null Types

These specify whether a value can be null or non-null. A type followed by an exclamation mark (!) is guaranteed to always have a value, which gives clients a reliable contract.

List Types

These define a collection of values of a specific type, written between square brackets, for example [String!]!.

Object Types

These represent complex objects with multiple fields. Each field has its own type, which allows objects to reference other objects and form a graph.

Enumeration Types

These define a specific set of allowed values, such as the roles ADMIN, STAFF, and GUEST.

Interface Types

These define a contract for the objects that implement them. An interface can specify common fields and functionality shared by multiple types.

Union Types

These represent a type that can be one of several possible object types, for example, a search result that may be either a User or a Booking.

Input Types

These are used as arguments for mutations to specify the data that will be created or updated. Unlike object types, they cannot have fields that return other objects.

Table 1 summarizes the type categories and gives an example of each one.

Category	Purpose	Example
Scalar	Built-in primitive value	String, Int, Boolean
Non-Null	Value cannot be null	String!
List	Collection of values	[String!]!
Object	Entity with several fields	type User { ... }
Enum	Fixed set of values	enum Role { ADMIN }
Interface	Shared contract	interface Node { id: ID! }
Union	One of several types	union SearchResult = User | Booking
Input	Arguments for mutations	input CreateUserInput { ... }

Table 1. Summary of GraphQL type categories

Queries

In GraphQL, queries are used to retrieve data from the server. A query describes the shape of the data you want to obtain and lets you specify which fields are needed. Queries are written in a structure similar to the shape of the desired response, as shown in Figure 3.

The "Query" type is special: it lists all the available queries that clients can run, along with the return type of each one.



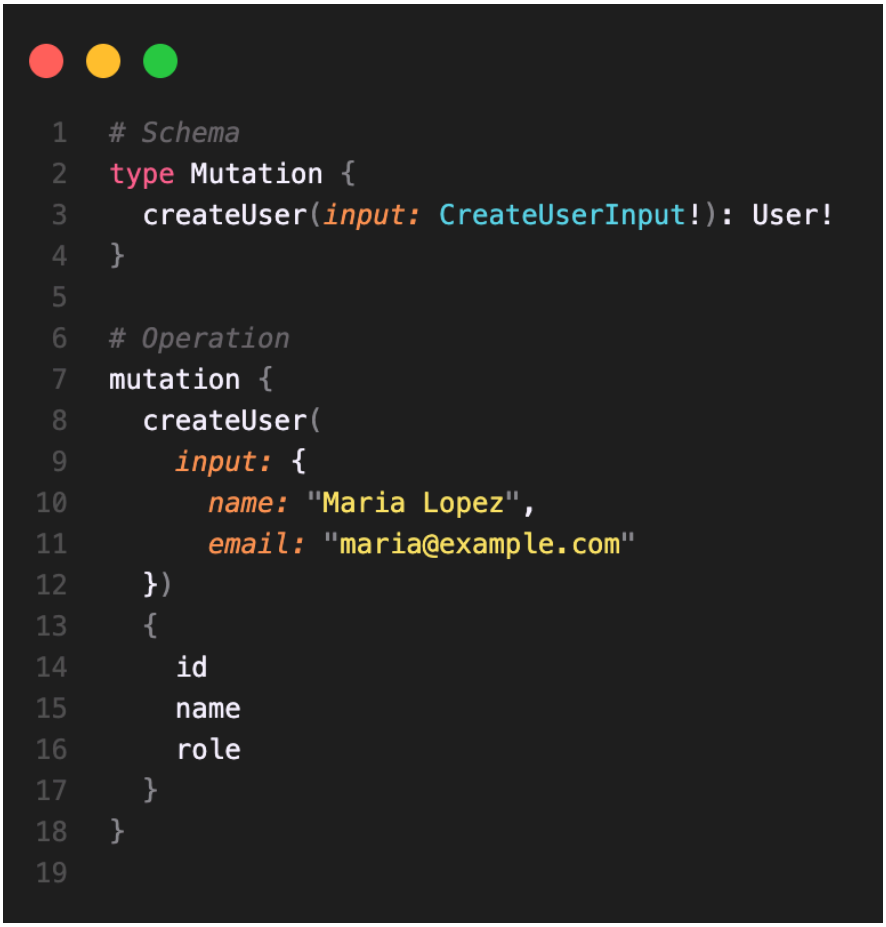
```graphql
query {
  user(id: "1") {
    name
    email
  }
}

# Response
{
  "data": {
    "user": {
      "name": "Maria Lopez",
      "email": "maria@example.com"
    }
  }
}
```

Figure 3. Query and its JSON response

Mutations

Mutations in GraphQL are used to modify data on the server. They allow data to be created, updated, or deleted. Mutations are similar to queries in structure, but they are defined with the mutation keyword. Mutations can take input arguments to specify the data to be modified, and they return the affected object so the client can update its state without making an additional request.



```
# Schema
type Mutation {
  createUser(input: CreateUserInput!): User!
}

# Operation
mutation {
  createUser(
    input: {
      name: "Maria Lopez",
      email: "maria@example.com"
  })
  {
    id
    name
    role
  }
}

```

Figure 4. Mutation that creates a user using an input type

Subscriptions

Subscriptions in GraphQL enable real-time data communication between the server and clients. They allow clients to subscribe to specific events or data changes and receive updates when those events occur. Subscriptions are defined similarly to queries and mutations, but they are prefixed with the subscription keyword. In Node.js, they are commonly delivered over WebSockets.

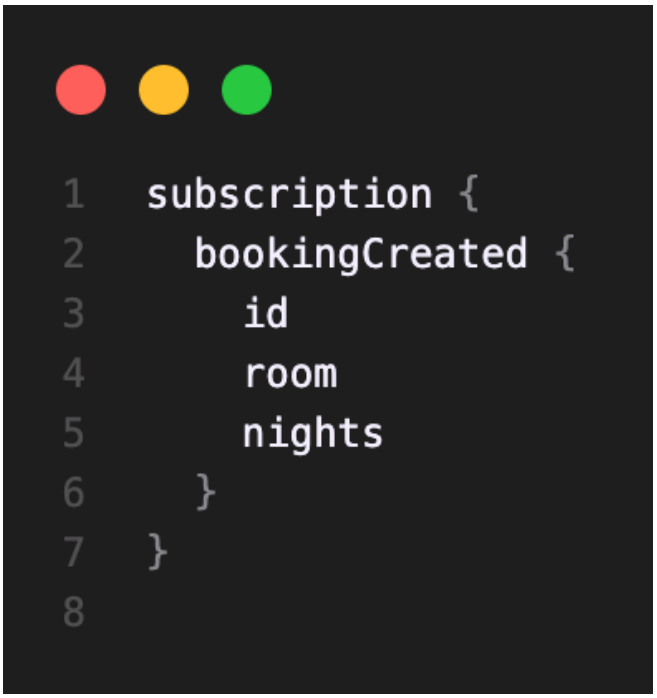


Figure 5. Subscription that notifies the client when a booking is created

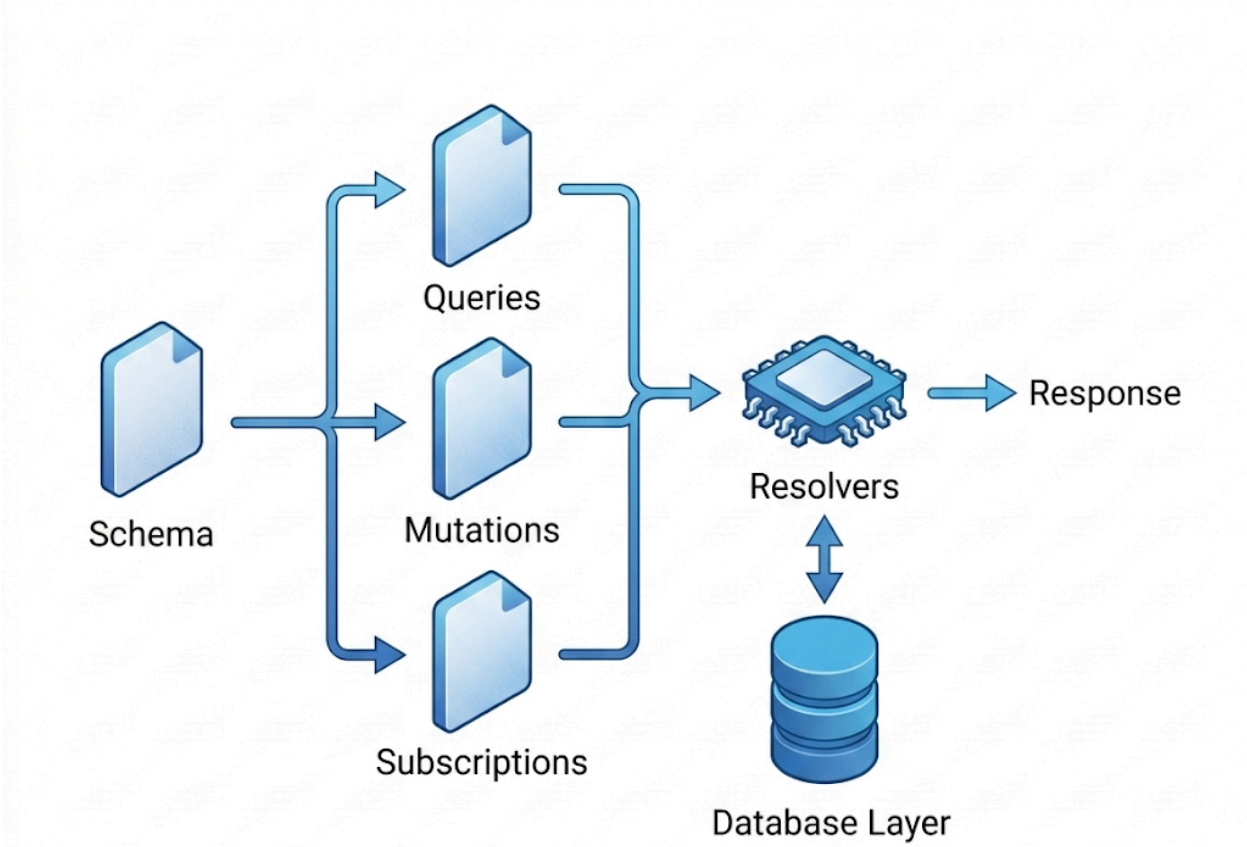


Figure 6. GraphQL schema structure detailing queries, mutations, and subscriptions processed through resolvers and the database layer

Variables and Aliases

Variables and aliases in GraphQL improve code organization and the handling of queries on the client side.

Variables

Variables are client-defined parameters in a query that allow the same query to be customized and reused with different values. They provide dynamic values for the arguments of query fields, which allows greater flexibility and efficiency in communication between the client and the server. When defining variables, their type is specified and, optionally, a default value is assigned. This approach promotes modularity, readability, and query security, while encouraging good development practices by separating query logic from its specific values, as shown in Figure 6.



Figure 7. Query that uses a variable and its JSON variables object

Aliases

Aliases allow clients to rename the fields returned in a query according to their specific needs. This is especially useful when requesting multiple fields of the same type in a single query, since it avoids ambiguities and name collisions. Aliases are defined by preceding the desired field with a custom name followed by a colon (:), which makes it easier for the client to distinguish and process the received data.



```
query {
  admin: user(id: "1") { name }
  guest: user(id: "2") { name }
}

# Response
{
  "data": {
    "admin": { "name": "Maria Lopez" },
    "guest": { "name": "John Smith" }
  }
}
```

Figure 8. Two users requested in the same query using aliases

Custom Scalars

In GraphQL, custom scalars are user-defined scalar data types that make it possible to represent specific data more semantically and precisely in a schema. Although the specification includes default scalar types such as Int, Float, String, Boolean, and ID, it is often necessary to support custom atomic data types (such as Date or Email) or to create custom versions of existing types that perform additional validations.

Unlike enumerations, which limit their values to a predefined list of strings, custom scalars can represent a wider variety of atomic data with custom validation. In Node.js, a custom scalar is implemented with the GraphQLScalarType class, as shown in Figure 8: serialize converts the value sent to the client, while parseValue and parseLiteral validate the value received from it.

```
# Schema
scalar Date

type Booking {
  id: ID!
  checkIn: Date!
}

import { GraphQLScalarType, Kind } from 'graphql';

const DateScalar = new GraphQLScalarType({
  name: 'Date',
  serialize: (value) => value.toISOString().slice(0, 10),
  parseValue: (value) => new Date(value),
  parseLiteral: (ast) =>
    ast.kind === Kind.STRING ? new Date(ast.value) : null,
});

```

Figure 9. Custom Date scalar declared in the schema and implemented in Node.js

Dynamic Nesting

Dynamic nesting in GraphQL refers to the ability to define complex relationships between data types flexibly and efficiently. It allows queries to specify the depth and structure of the data they want to retrieve, which makes it easy to fetch related data intuitively and without multiple requests, as shown in Figure 9.



Figure 10. Nested query that retrieves a user together with its bookings

With dynamic nesting, clients can access related data efficiently, which improves query performance and reduces network overhead. This makes GraphQL especially well suited to applications that require hierarchical and complex data retrieval, such as web and mobile applications.

Dynamic nesting in GraphQL refers to the ability to define complex relationships between data types flexibly and efficiently. This allows GraphQL queries to specify the depth and structure of the data they want to retrieve, which makes it easy to fetch nested data intuitively and without multiple requests. The posts field inside user in Figure 2 is an example of nesting.

With dynamic nesting, clients can access related data efficiently, which improves query performance and reduces network overhead. This makes GraphQL especially well suited to applications that require hierarchical and complex data retrieval, such as web and mobile applications.

Building a GraphQL Server with Node.js

This section applies the concepts above in a minimal server built with Apollo Server, one of the most widely used GraphQL libraries for Node.js. The server needs only two pieces: the schema (typeDefs), which describes the API, and the resolvers, which are the functions that return the data for each field.

Project Setup

Create a new project, enable ES modules, and install the required packages, as shown in Figure 10. Node.js and npm must be installed beforehand.

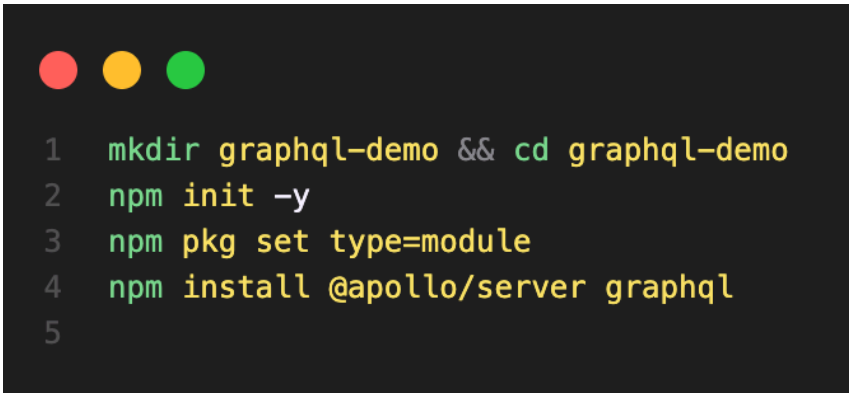


Figure 11. Commands to create the project and install the dependencies

Schema and Resolvers

The following file defines a schema with a single query and its resolver, and starts the server on port 4000. Each resolver receives the parent object, the arguments of the field, and a context shared by the whole request, which is typically used for the database connection or the authenticated user.

```
import { ApolloServer } from '@apollo/server';
import { startStandaloneServer } from '@apollo/server/standalone';

const typeDefs = `#graphql
  type Query {
    hello: String!
  }
`;

const resolvers = {
  Query: {
    hello: () => 'Hello, GraphQL!',
  },
};

const server = new ApolloServer({ typeDefs, resolvers });

const { url } = await startStandaloneServer(server, {
  listen: { port: 4000 },
});

console.log(`Server ready at ${url}`);

```

Figure 12. Minimal GraphQL server using Apollo Server (index.js)

Nested fields are resolved by their own resolvers. In Figure 12, the bookings field of User is only executed when a client actually asks for it, which is what makes dynamic nesting efficient.

```
const resolvers = {
  Query: {
    user: (_, { id }, ctx) => ctx.db.users.findById(id),
  },
  User: {
    bookings: (user, _, ctx) => ctx.db.bookings.findByUser(user.id),
  },
};
```

Figure 13. Resolvers for a root query and a nested field

Testing the Server

Run the server with node index.js and open http://localhost:4000 in the browser. In development, Apollo Server provides an interactive explorer where queries can be written, executed, and documented against the schema without any additional tools.

GraphQL vs REST API

GraphQL and REST can both power a modern application, but they distribute control differently. In REST, the server defines the shape of each response and the client may need several requests to build a screen. In GraphQL, the client defines the shape of the response and receives it in a single request. Table 2 compares both approaches, and Figure 13 shows the same use case in each one.

Aspect	REST API	GraphQL
Data control	Server	Client
Data returned	Unnecessary data (all the data)	Necessary data (only what is requested)
Requests	N requests	1 request
Endpoints	X endpoints	1 endpoint
Teams	Dependency between teams	Independence between teams
Schema and documentation	External (e.g., OpenAPI)	Built-in and introspectable

Table 2. Comparison between REST API and GraphQL

In a Node.js application, a GraphQL API is built from two main elements: the schema, which describes the data that can be requested, and the resolvers, which are the functions that obtain the data for each field of the schema. When a query arrives, the server validates it against the schema and then executes the resolvers needed to build the response. Figure 3 summarizes this flow.

Figure 3. Lifecycle of a GraphQL request in a Node.js server

There are several libraries to build GraphQL servers in Node.js. Figure 4 shows a minimal server created with Apollo Server, one of the most widely used options. It requires the @apollo/server and graphql packages, which can be installed with npm, and a project configured to use ES modules.

Figure 4. Minimal GraphQL server in Node.js using Apollo Server

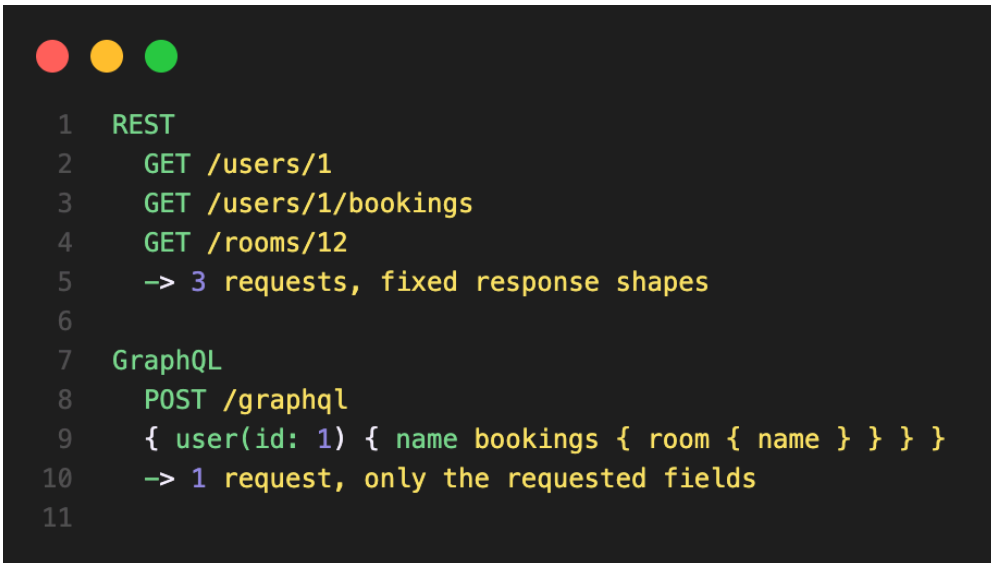


Figure 14. Same use case implemented with REST and with GraphQL

Best Practices

The following recommendations help keep a GraphQL API secure, fast, and maintainable as it grows:

- Design the schema around the business domain, not around database tables.
- Use non-null types deliberately so the schema reflects real guarantees.
- Validate inputs with input types and custom scalars.
- Paginate lists instead of returning unbounded collections.
- Limit query depth and complexity to protect the server from abusive queries.
- Avoid the N+1 problem by batching data access, for example with DataLoader.
- Return meaningful errors and handle authentication and authorization in the context and resolvers.

Conclusion

GraphQL offers a precise, strongly typed, and flexible way to build APIs. Its single endpoint, client-driven queries, and self-documenting schema reduce network overhead and decouple front-end and back-end teams. Combined with Node.js, it provides a productive foundation for building modern web and mobile applications.

To create custom versions of existing types that perform additional validations.

Custom scalars provide this flexibility by allowing developers to define their own scalar types to fit the specific needs of their applications. Unlike enumerations, which limit their values to a predefined list of strings, custom scalars can represent a wider variety of atomic data with custom validations.

GraphQL gives clients precise control over the data they receive, reduces the number of requests, and offers a single, strongly typed endpoint described by a schema. Combined with Node.js, it makes it possible to build flexible and maintainable APIs. The concepts presented in this manual (types, queries, mutations, subscriptions, variables, aliases, custom scalars, and dynamic nesting) are the foundation for building and consuming GraphQL APIs.